

CASE STUDY 2

Introduction

The main goal of this project was to understand how deep learning based object detection and multi object tracking work in real video scenes. I wanted to see how different models behave under different conditions and which combinations perform better for specific types of video sequences.

Instead of only running a model and reporting numbers, I focused on comparing results across multiple MOT17 sequences. Each sequence has its own challenges, such as occlusion, crowd density, camera movement, and missed detections. Because of these differences, one method may work better in one sequence but worse in another.

To measure performance, I used common multi-object tracking metrics such as MOTA, HOTA, and IDF1. By analyzing these results, I tried to understand which detector tracker combination performs better and why.

The purpose of this study was not only to get high metric scores, but also to understand the behavior of the system and explain where it works well and where it fails.

Model and Tracking Pipeline

In this project, I used two different deep learning detection models and two different tracking algorithms. I combined them to build a four-way pipeline for comparison. The detection models were YOLOv8 and YOLOv11, and the tracking algorithms were ByteTrack and BotSORT from the Ultralytics framework.

By combining each detector with each tracker, I created four different setups:

YOLOv8 + ByteTrack

YOLOv8 + BotSORT

YOLOv11 + ByteTrack

YOLOv11 + BotSORT

This allowed me to clearly see how both the detector and the tracker affect the final tracking performance.

To go one step further than using default settings, I created custom versions of ByteTrack and BotSORT. I fine-tuned the tracking parameters by adjusting values such as the high-confidence threshold and low-confidence threshold. The goal was to reduce ID switches, improve track stability, and get more consistent tracking results.

For evaluation, I selected the following MOT17 sequences:

MOT17-02

MOT17-04

MOT17-05

MOT17-09

MOT17-10

MOT17-11

MOT17-13

These sequences were chosen because they have different levels of crowd density, occlusion, and scene difficulty. This helped me compare how each method performs under different conditions.

About the MOT17 Dataset

Although MOT17 provides different detection versions (DPM, SDP, FRCNN), in this project I used YOLOv8 and YOLOv11 as the detector. Therefore, the original detection type of the sequence does not affect my results. The selected sequences differ mainly in scene complexity, crowd density, and motion characteristics.

Implementation Process

I did not start directly with the final four ways pipeline. First, I implemented a simpler version using YOLOv8 Nano with the default ByteTrack algorithm. The goal was to test the basic detection and tracking pipeline and observe how it performs on MOT17 sequences.

However, in crowded scenes, the results were not very good. The tracker was losing people more often, and there were more ID switches. I was not completely sure whether the problem was caused by the tracker or because YOLOv8 Nano was too small for complex scenes.

To improve performance, I switched to YOLOv8 Large with ByteTrack. After this change, detection quality improved, especially in crowded areas, and tracking became more stable. Missed detections decreased, and tracking metrics showed better results.

After discussing the project with my instructor, the direction became clearer. The goal was not only to use existing libraries, but also to understand how the tracking algorithms work internally and how parameter changes affect performance. Instead of only running default configurations, I started analyzing the thresholds used in ByteTrack and BotSORT.

Based on this discussion, I created custom configuration files for both ByteTrack and BotSORT. I fine-tuned important tracking parameters such as high confidence threshold, low confidence threshold, and matching related thresholds. These custom YAML files were then used directly inside the tracking pipeline.

This allowed me to observe how small parameter changes affect metrics like MOTA and IDF1. In some sequences, adjusting thresholds reduced ID switches and improved ID consistency, while in others the effect was limited. This helped me better understand how tracking behavior changes depending on scene difficulty.

Finally, I expanded the project to compare two detectors (YOLOv8 and YOLOv11) and two trackers (ByteTrack and BotSORT), creating four different experimental setups. I evaluated their performance using tracking metrics and visualized the results using bubble graphs to clearly compare MOTA and IDF1 values across sequences.

Custom Tracker Configuration

Instead of using the default tracker settings, I created custom YAML configuration files for both BotSORT and ByteTrack. The goal was to fine-tune the tracking thresholds and observe how they affect tracking stability and ID consistency.

Below are the custom parameters I used.

```
1 tracker_type: botsort
2 track_high_thresh: 0.4
3 track_low_thresh: 0.1
4 new_track_thresh: 0.5
5 track_buffer: 60
6 match_thresh: 0.85
7 fuse_score: True
8 gmc_method: sparseOptFlow
9 proximity_thresh: 0.5
10 appearance_thresh: 0.8
11 with_reid: False
12
```

For BotSORT, I increased the `track_high_thresh` to 0.4 to make sure that only confident detections are used to start strong tracks. I kept `track_low_thresh` lower (0.1) to allow recovery in difficult frames. I also adjusted the `match_thresh` to 0.85 to make matching stricter and reduce ID switches.

The `track_buffer` was set to 60 to allow tracks to survive short-term occlusions. I couldn't configure the ReID so I disabled it in order for the system to work .

```
1 tracker_type: bytetrack
2 track_high_thresh: 0.25
3 track_low_thresh: 0.1
4 new_track_thresh: 0.3
5 track_buffer: 60
6 match_thresh: 0.8
7 fuse_score: True
8
```

For ByteTrack, I used a lower `track_high_thresh` (0.25) compared to BotSORT. This allows more detections to be considered for tracking, which can help in crowded scenes but may increase false positives. The `match_thresh` was set to 0.8 to balance between stability and flexibility. Finally `track_buffer` was set to 60 again to allow tracks to survive short occlusions.

Evaluation and Metrics

Each sequence contains 416 frames each and they are approximately about 20 to 30 seconds each. After generating the overlay videos for all four pipelines, I tried to compare the results visually.

However, it was difficult to clearly understand the performance differences only by looking at the videos. Small differences in tracking stability, ID switches, or missed detections are not always easy to detect with the human eye. Because of this, I realized that visual comparison alone was not enough.

After another discussion with my instructor, we decided that I should evaluate the results using official multi object tracking metrics. I researched common MOT metrics such as MOTA, HOTA, MOTP, and IDF1. These metrics measure different aspects of tracking performance, such as accuracy, identity consistency, and localization precision.

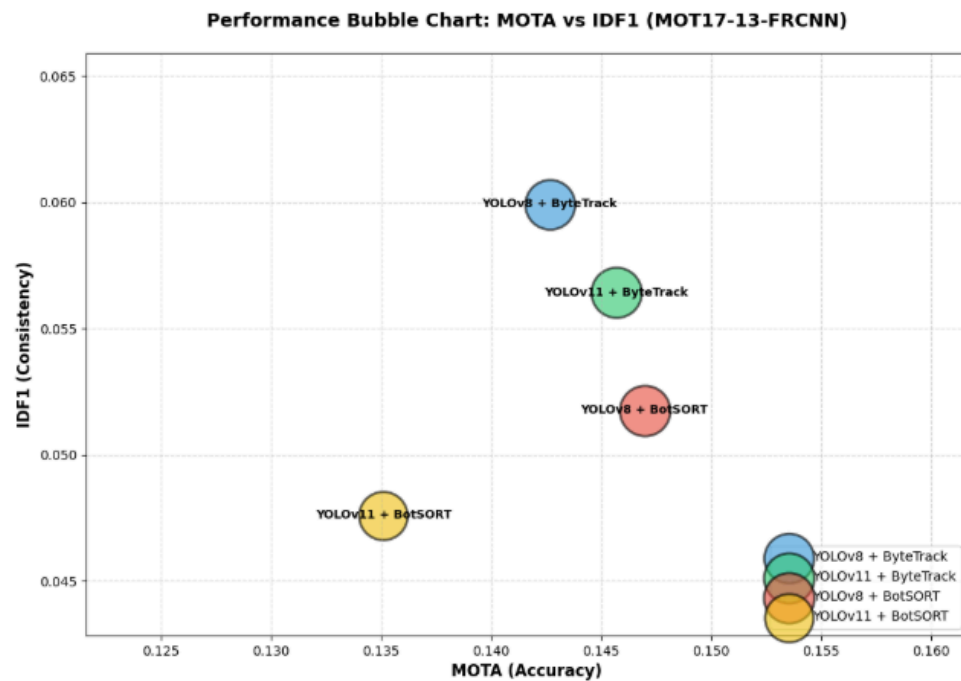
Finally, I decided to focus mainly on MOTA and IDF1:

MOTA measures overall tracking accuracy by considering false positives, missed detections, and ID switches. whereas IDF1 measures how consistent the identity tracking is over time.

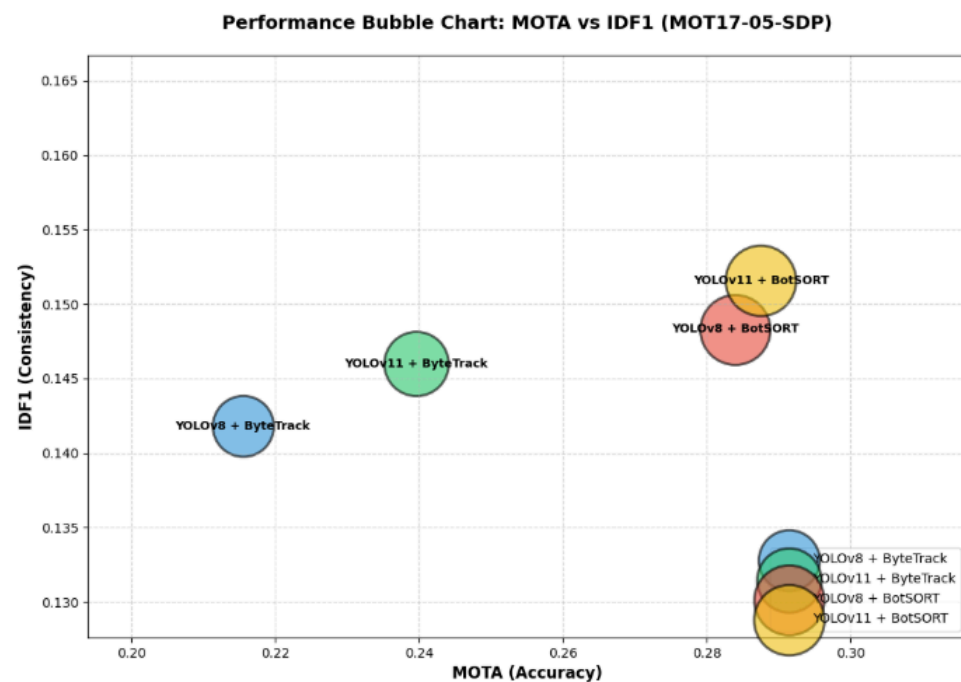
I selected these two metrics because together they provide a clear view of both detection quality and identity consistency. Using these scores, I was able to compare the four pipelines

more objectively and determine which detector and tracker combination performs better for each sequence.

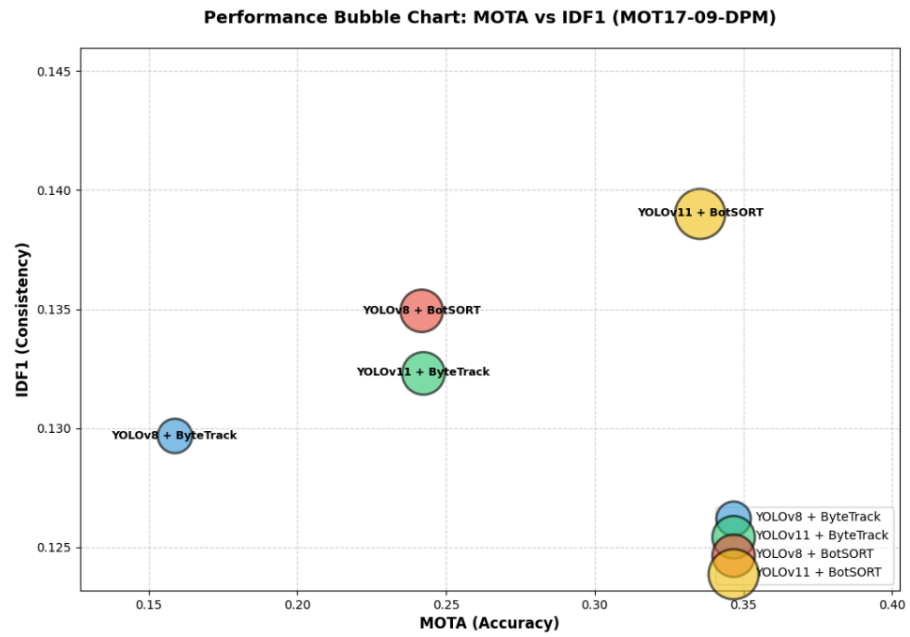
Here below I have the bubble charts of the video sequences I implemented:



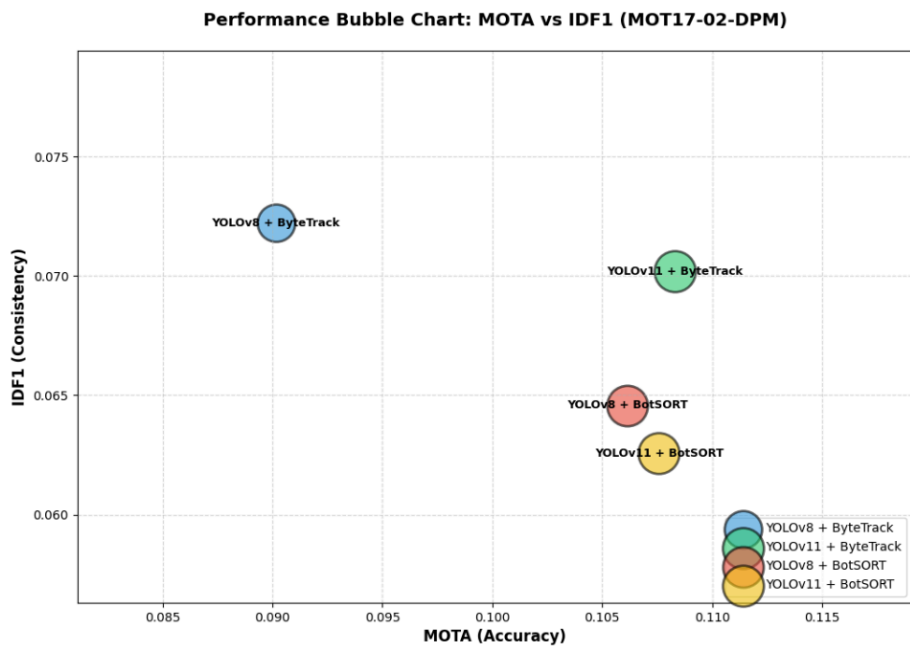
bubble chart for the MOT17-13-FRCNN video sequence



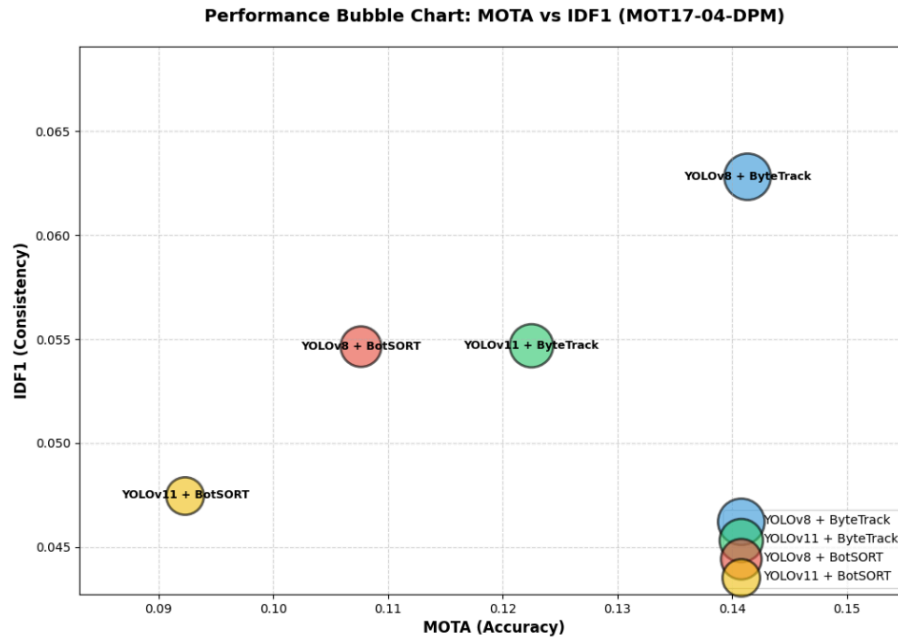
bubble chart for the MOT17-05-SDP video sequence



bubble chart for the MOT17-09-DPM video sequence



bubble chart for the MOT17-02-DPM video sequence



bubble chart for the MOT17-04-DPM video sequence

Results and Analysis

Based on these graphs, we can confidently say that YOLOv11 is positioned to the right of YOLOv8. This indicates that YOLOv11 has a higher MOTA value, meaning better accuracy compared to YOLOv8. Especially when looking at sequences 05, 09, and 11, this pattern becomes very clear. From this, we can say that the model quality of the pipeline directly affects the MOTA score. YOLOv11 improves detection quality, so fewer detections are missed, which results in a better MOTA value.

When we compare the trackers, BotSORT pipelines are positioned slightly higher on the graphs. This indicates better ID consistency based on the IDF1 metric. Therefore, we can say that the tracker choice mainly affects IDF1. This is because in BotSORT we used stricter matching with a higher threshold parameter (`match_thresh`), which reduced ID switches and improved identity consistency.

Another important observation from the graphs is that the overall metric values change significantly between sequences. Some sequences such as MOT17-11 show higher IDF1 and MOTA values, while sequences like MOT17-02 and MOT17-04 have much lower scores. This shows that scene difficulty, crowd density, and occlusion levels affect tracking performance. In more complex scenes, identity switches and missed detections increase, which lowers both MOTA and IDF1.

In some sequences, the pipeline with the highest MOTA does not always achieve the highest IDF1. This shows that better detection accuracy does not automatically guarantee better identity consistency. Detection quality and tracking stability are related, but they are still separate factors in the pipeline.

Overall, YOLOv11 combined with BotSORT is often positioned in the upper right area of the graphs. This means it provides a balanced improvement in both accuracy and identity consistency. However, the best pipeline is not exactly the same for every sequence, which confirms that different scenes require different strengths from the tracking system.